

**National Institute of Technology Calicut**  
**Department of Computer Science and Engineering**

**CS4086E: Systems Programming Lab**

Introduction to Make

Practice Questions

Date: 23, Jan.'26

**Instructions**

- It may be noted that no specific functions/programs are provided to the questions
- Students can decide on the program file; left to the imagination of the students
- You may make use of the source codes that you worked in the 'Program Design lab' or 'Data Structures lab'
- Reference Materials:
  - Chapter No. 4 in the uploaded ebook authored by *Rehman and Paul*
  - Chapter No. 3 in the uploaded ebook authored by *Jack-Benny Persson*

**Beginners**

1. Write a Makefile that creates a static library libdata.a using x.o and y.o.  
Input: x.c y.c  
Output: libdata.a
2. Write a Makefile that uses variables to store the compiler name and compilation flags while compiling a C source file test.c to generate an executable named test.  
Input: test.c  
Output: test
3. Write a Makefile for a project containing main.c and helper.c that compiles both into an executable program. Use variables for compiler and flags. Include explicit rules for object files and a comprehensive clean target.  
Input: main.c, helper.c  
Output: program executable
4. A program hello is built from a single source file hello.c. The source file includes a header file hello.h. Both files are in the same directory. Write a Makefile to compile hello.c and generate the executable hello.  
Input: hello.c, hello.h, Makefile  
Output: produce the executable file named hello
5. A program calc is built from a single source file calc.c. The source file includes a header file calc.h. Both files are located in the same directory. Write a Makefile to compile calc.c and generate the executable calc.  
Input: calc.c, calc.h  
Output: creation of the object file calc.o followed by the generation of the executable file calc.

**National Institute of Technology Calicut**  
**Department of Computer Science and Engineering**

6. Write a Makefile to compile a single C source file sum.c into an executable named sumprog. Use variables for the compiler and compiler flags.

**Input:** sum.c

**Output:** Executable file: sumprog

7. Write a Makefile that compiles two source files a.c and b.c into object files and then archives them into a static library named libab.a.

**Input:**

a.c

b.c

**Output:**

Object files generated: a.o, b.o

Static library generated: libab.a

8. Write a Makefile to compile welcome.c into an executable named 'welcome'. The Makefile must include a clean target that removes all generated files.

**Input:** welcome.c

**Output:**

Executable file welcome

make clean removes the executable

**Intermediate**

9. A project contains three source files x.c, y.c, and z.c. The files x.c and y.c are used together to build an executable named prog, while the file z.c is used only to generate an object file z.o. Write a Makefile such that running make builds the executable prog, and running make z builds only the object file z.o. You do not need to write any source files.

**Input:** x.c, y.c, z.c

**Output:**

Running make generates the executable prog

Running make z generates the object file z.o

10. A project contains three source files main.c, input.c, and output.c. The header files input.h and output.h are included where required. Write a Makefile that does the following:

Compiles each source file into its corresponding object file

- Links all object files into an executable named ioapp
- Uses variables for compiler and compilation flags

**Input:** main.c

input.c, input.h

output.c, output.h

**Output:** Object files generated: main.o, input.o, output.o

Executable file generated: ioapp

**National Institute of Technology Calicut**  
**Department of Computer Science and Engineering**

11. A directory contains multiple C source files. Write a Makefile that automatically compiles all .c files in the directory and links them into an executable named autoapp.

**Input:**

file1.c  
file2.c  
file3.c

**Output:**

Object files generated: file1.o, file2.o, file3.o  
Executable file generated: autoapp

12. Write a **Makefile** for a project containing two C source files **math.c** and **util.c** along with their header files **math.h** and **util.h**. The Makefile should:

- Compile each source file into its corresponding object file
- Link the object files to generate an executable named **calculator**
- Use variables for the compiler and compiler flags
- Include a **clean** target that removes all object files and the executable

**Input:**

math.c, util.c, math.h, util.h

**Output:**

Object files generated: math.o, util.o  
Executable file generated: calculator  
make clean removes all object files and the executable

**Advanced**

13. Build a Create a shift-by-1 encoding/decoding utility using both Static/Dynamic Library. A file compression utility provides operations to:

- Shift-by-1 Encode
- Shift-by-1 Decode

These operations should be compiled into a library (*libcompress*) and used by a main program to provide user commands.

Write a Makefile that:

- Compiles encode and decode modules into object files.
- Archives them into a library (*libencodec*).
- Links the library with the main program to create an executable (*encodec*).
- Refer *Appendix-A* for the Shift-by-1 Encode and Decode source code

**National Institute of Technology Calicut**  
**Department of Computer Science and Engineering**

14. A data processing system requires efficient sorting functions to handle large datasets. The system should support 'Bubble Sort'. The sorting function should be compiled into a library (libsortutils).

Write a Makefile that:

- Compiles sorting algorithm into object files.
- Creates a shared library (*libsortutils*) from these object files.
- Compiles the main program separately and links it to libsortutils.

Appendix-A

**shift\_encoding.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void encode_file(const char *input, const char *output) {
    FILE *fin = fopen(input, "r");
    FILE *fout = fopen(output, "w");
    if (!fin || !fout) {
        perror("File error");
        return;
    }
    char ch;
    while ((ch = fgetc(fin)) != EOF) {
        fputc(ch + 1, fout);          // Simple shift encoding
    }
    fclose(fin);
    fclose(fout);
}
```

**Shift\_decoding.c**

```
#include <stdio.h>
#include <stdlib.h>

void decode_file(const char *input, const char *output) {
    FILE *fin = fopen(input, "r");
    FILE *fout = fopen(output, "w");
    if (!fin || !fout) {
        perror("File error");
        return;
    }
    char ch;
    while ((ch = fgetc(fin)) != EOF) {
        fputc(ch - 1, fout); // Reverse shift encoding
    }
    fclose(fin);
    fclose(fout);
}
```

